



ALLIANCE
UNIVERSITY
Private University established in Karnataka State by Act No. 84 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

NAAC
GRADE A+
ACCREDITED UNIVERSITY

Bachelor of Computer Applications

Semester – V

Deep Learning

project No: 9

Title: Optimizer Comparison Using a Neural Network

Submitted By:

Name: R. Nitheesh

Reg. No.: 2411021240047

Date: 9-10-2026

Submitted To:

Faculty Name: Nagasree

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru – 562106

project No: 9

Introduction

Optimization algorithms play an important role in training neural networks. They update the model's weights and biases to minimize the loss function.

In this project, a simple neural network is built using the Iris dataset. Two optimization algorithms, Stochastic Gradient Descent (SGD) and Adam, are used to train the same neural network. Their training loss, accuracy, and final classification performance are compared.

Objectives

- Build a neural network using the Iris dataset.
- Train the network using SGD.
- Train the network using Adam.
- Compare training loss and accuracy.
- Evaluate the classification performance of both optimizers.

Import Required Libraries

Explanation

The following libraries are used for loading the dataset, preprocessing the data, creating the neural network, training the models, and evaluating their performance.

Python Code

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import accuracy_score, classification_report
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
from tensorflow.keras.optimizers import SGD, Adam
```

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD, Adam
```

Load the Iris Dataset

The Iris dataset contains 150 samples belonging to three different flower species:

- Setosa
- Versicolor
- Virginica

Each sample contains four input features:

- Sepal length
- Sepal width
- Petal length
- Petal width

```
# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

print("Dataset Shape:", X.shape)
print("Target Shape:", y.shape)
print("Class Names:", iris.target_names)

Dataset Shape: (150, 4)
Target Shape: (150,)
Class Names: ['setosa' 'versicolor' 'virginica']
```

Split and Standardize the Dataset

Explanation

The dataset is divided into training and testing sets. Standardization is applied so that all input features have a similar scale.

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Standardize features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

... Training samples: 120
Testing samples: 30
```

Build the Neural Network

Explanation

A simple feed-forward neural network is created using Keras.

The network contains:

- Input layer: 4 neurons
- Hidden layer: 16 neurons with ReLU activation
- Hidden layer: 8 neurons with ReLU activation
- Output layer: 3 neurons with Softmax activation

The three output neurons represent the three Iris classes.

```
def create_model():  
    model = Sequential([  
        Dense(16, activation='relu', input_shape=(4,)),  
        Dense(8, activation='relu'),  
        Dense(3, activation='softmax')  
    ])  
  
    return model
```

Train the Model Using SGD

Explanation

SGD stands for Stochastic Gradient Descent. It updates the model parameters based on gradients calculated from the training data.

```
# Create SGD model  
model_sgd = create_model()  
  
# Compile model using SGD  
model_sgd.compile(  
    optimizer=SGD(learning_rate=0.01),  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
# Train model  
history_sgd = model_sgd.fit(  
    X_train,  
    y_train,  
    epochs=100,  
    batch_size=16,  
    validation_split=0.2,  
    verbose=0  
)  
  
print("SGD training completed.")  
  
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an  
super().__init__(activity_regularizer=activity_regularizer, **kwargs)  
SGD training completed.
```

Train the Model Using Adam

Adam stands for Adaptive Moment Estimation. It combines ideas from momentum and adaptive learning rates to update the neural network parameters.

```
# Create Adam model
model_adam = create_model()

# Compile model using Adam
model_adam.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

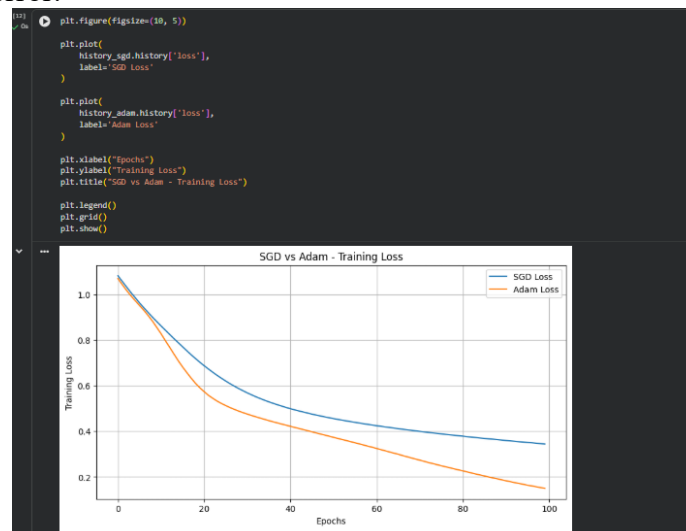
# Train model
history_adam = model_adam.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=16,
    validation_split=0.2,
    verbose=0
)

print("Adam training completed.")

Adam training completed.
```

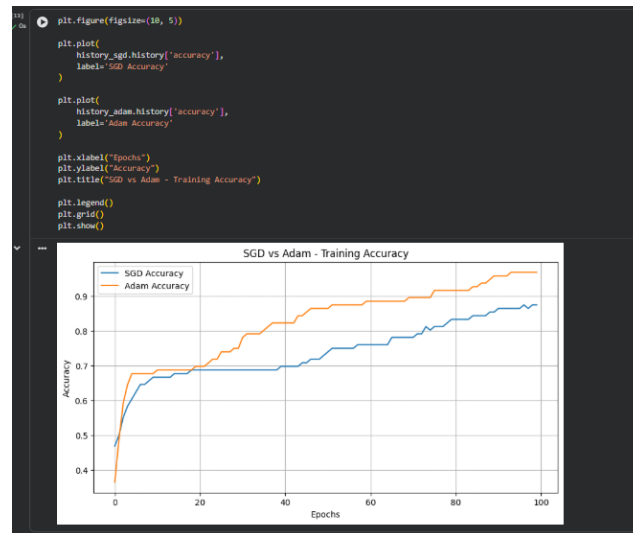
Compare Training Loss

The training loss of SGD and Adam is plotted on the same graph to compare how quickly each optimizer reduces the error.



Compare Training Accuracy

The training accuracy of both models is plotted to compare how their classification performance changes during training.



Evaluate SGD Model

The trained SGD model is evaluated using the test dataset.

```
# Predict using SGD
y_pred_sgd = model_sgd.predict(X_test)

# Convert probabilities to class labels
y_pred_sgd_classes = np.argmax(y_pred_sgd, axis=1)

# Calculate accuracy
sgd_accuracy = accuracy_score(y_test, y_pred_sgd_classes)

print("SGD Test Accuracy:", sgd_accuracy)
print("\nSGD Classification Report:")
print(classification_report(
    y_test,
    y_pred_sgd_classes,
    target_names=iris.target_names
))
```

```
1/1 ————— 0s 339ms/step
SGD Test Accuracy: 0.8

SGD Classification Report:
              precision    recall  f1-score   support

   setosa         1.00        1.00        1.00        10
  versicolor       1.00        0.40        0.57        10
   virginica       0.62        1.00        0.77        10

   accuracy         0.88        0.80        0.80        30
  macro avg         0.88        0.80        0.78        30
 weighted avg         0.88        0.80        0.78        30
```

Evaluate Adam Model

```
▶ # Predict using Adam
y_pred_adam = model_adam.predict(X_test)

# Convert probabilities to class labels
y_pred_adam_classes = np.argmax(y_pred_adam, axis=1)

# Calculate accuracy
adam_accuracy = accuracy_score(y_test, y_pred_adam_classes)

print("Adam Test Accuracy:", adam_accuracy)
print("\nAdam Classification Report:")
print(classification_report(
    y_test,
    y_pred_adam_classes,
    target_names=iris.target_names
))
```

```
... 1/1 ----- 0s 177ms/step
Adam Test Accuracy: 0.9666666666666667

Adam Classification Report:
              precision    recall  f1-score   support

   setosa         1.00        1.00        1.00        10
  versicolor       1.00        0.90        0.95        10
   virginica       0.91        1.00        0.95        10

 accuracy         0.97        0.97        0.97        30
  macro avg       0.97        0.97        0.97        30
 weighted avg     0.97        0.97        0.97        30
```

Compare Final Accuracy

The final test accuracy values of both optimizers are displayed together for comparison

```
▶ print("Optimizer Comparison")
print("-----")
print(f"SGD Test Accuracy : {sgd_accuracy:.4f}")
print(f"Adam Test Accuracy : {adam_accuracy:.4f}")
```

```
... Optimizer Comparison
-----
SGD Test Accuracy : 0.8000
Adam Test Accuracy : 0.9667
```

Analysis

In this project, a simple neural network was developed using the Iris classification dataset. The same network architecture and dataset were used to compare two optimization algorithms: Stochastic Gradient Descent (SGD) and Adam.

The dataset was divided into training and testing sets, and the input features were standardized before training. Two separate neural network models were created so that each optimizer could be evaluated independently.

The SGD model was trained using a learning rate of 0.01, while the Adam model was trained using a learning rate of 0.001. Both models were trained for 100 epochs using a batch size of 16.

The training loss and accuracy graphs were used to observe the learning behavior of the two optimizers. The test accuracy and classification reports were then used to evaluate their classification performance on unseen data.

The experiment demonstrates that different optimization algorithms can produce different training behavior even when the same neural network architecture and dataset are used. The final accuracy values obtained from the experiment provide a quantitative comparison between SGD and Adam.

Conclusion

This project successfully implemented a neural network for classifying the Iris dataset and compared two commonly used optimization algorithms, SGD and Adam.

Both optimizers were used to train the same neural network architecture. Their training loss, training accuracy, and test classification performance were evaluated.

The experiment showed how the choice of optimizer can affect the training behavior and convergence of a neural network. SGD updates parameters using gradient-based steps, while Adam uses adaptive estimates of gradients and their moments.

By comparing the loss and accuracy graphs along with the final test accuracies, the differences between SGD and Adam could be observed clearly.

Overall, this project provided practical experience with neural network optimization and demonstrated the importance of optimizers in Machine Learning model training.

